

Technical Report

1. Problem Recap & SMC Context

Problem Title:

Smart Health Monitoring & Decision Support System for Solapur Municipal Corporation

Problem Summary:

- Health data across SMC is fragmented across ASHA workers, hospitals, and laboratories
 - Lack of real-time monitoring and centralized data system
 - Delayed detection of disease outbreaks
 - No ward-level actionable health intelligence
-

Current Challenges in SMC:

- Disconnected reporting systems and manual data handling
 - Limited visibility of real-time health conditions at ward level
 - Slow response to emerging public health risks
 - Difficulty in identifying high-risk zones and vulnerable populations
-

Impact:

This results in **delayed decision-making, inefficient resource allocation, and increased public health risk**

2. Objectives & Implementation Mapping

Objective	Status	Evidence (Modules / Components)
Real-time ingestion of ASHA and hospital data	Prototype Implemented	ASHA Field Reporting App, Hospital Portal
Ward-level surveillance and monitoring	Prototype Implemented	Government Dashboard (charts, ward-level views)
Municipal command dashboard for decision-making	Implemented	Municipal Health Command Dashboard
Citizen interface for advisories and services	Implemented	Citizen Health App
Alert generation and early disease detection	Prototype Implemented	Alert Module, Python-based Prediction Engine
Secure and structured data handling	Prototype Implemented	Node.js Backend, PostgreSQL Database

SAMVED-2026 prototype submission format

3. Constraints & Handling Mapping

Constraint	Type	Handling Approach
Intermittent field connectivity	Technical / Operational	Implemented offline-first Flutter app using local SQLite storage with auto-sync on network availability
Data privacy & compliance	Legal / Technical	Secured APIs using TLS, JWT authentication, and Role-Based Access Control (RBAC) with audit logging
Data integrity (field reporting accuracy)	Operational / Technical	Enabled GPS-based validation using geospatial checks (PostGIS) to ensure reports originate from correct wards
Heterogeneous hospital systems	Technical	Built unified API Gateway with FHIR-based standardization for structured and interoperable data
Low-end device constraints	Hardware / Technical	Optimized mobile app performance using efficient state management, lazy loading, and lightweight UI rendering

SAMVED-2026 prototype submission format

4. System Architecture Subsystems

- Architecture Diagram

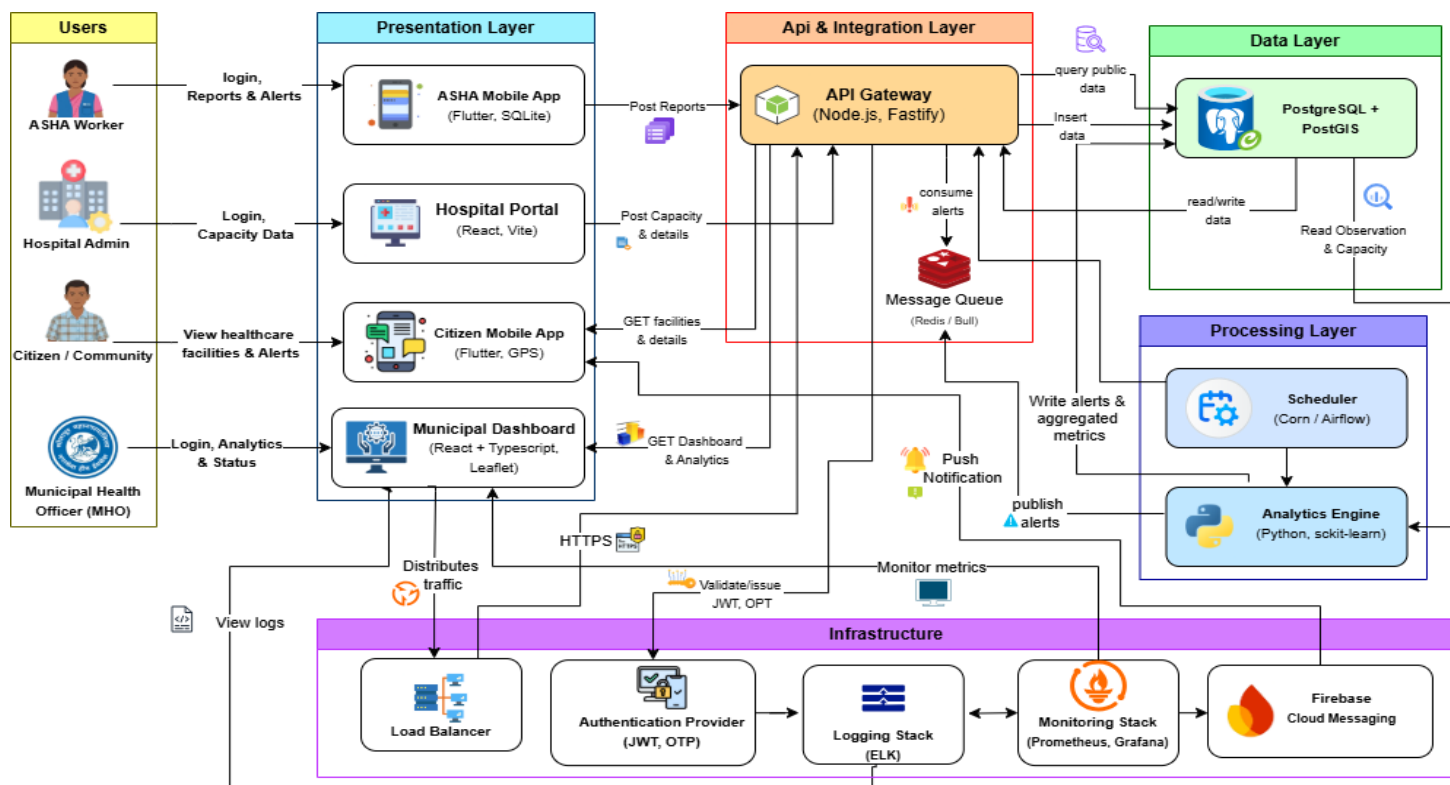


Figure 1: Complete Arogya-SMC System Architecture – *Integrated View of All Subsystems and Data Flows*

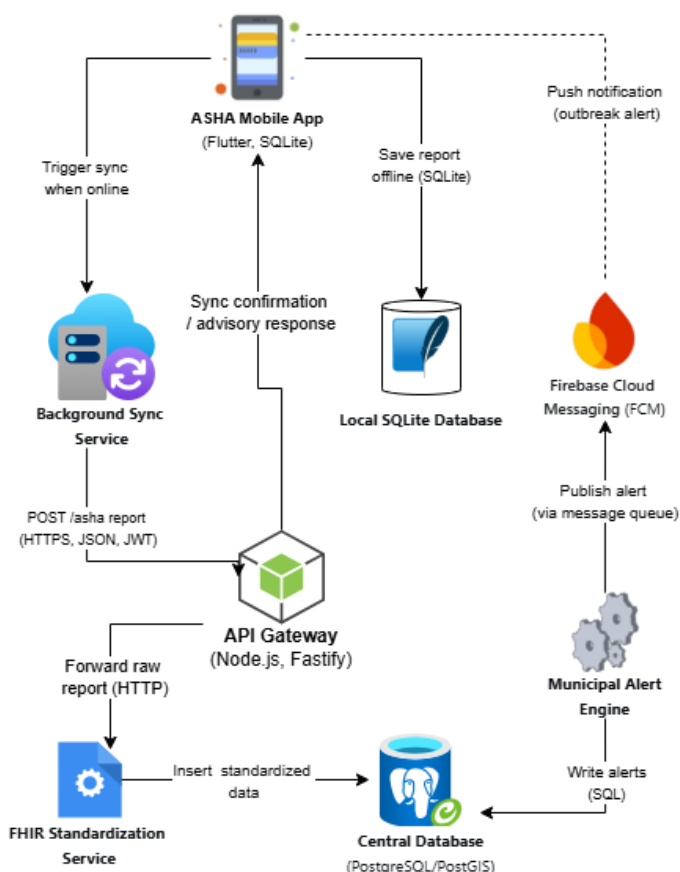


Figure 2: ASHA Field Worker Subsystem – *Offline Data Capture, Synchronization, and Alert*

SAMVED-2026 prototype submission format

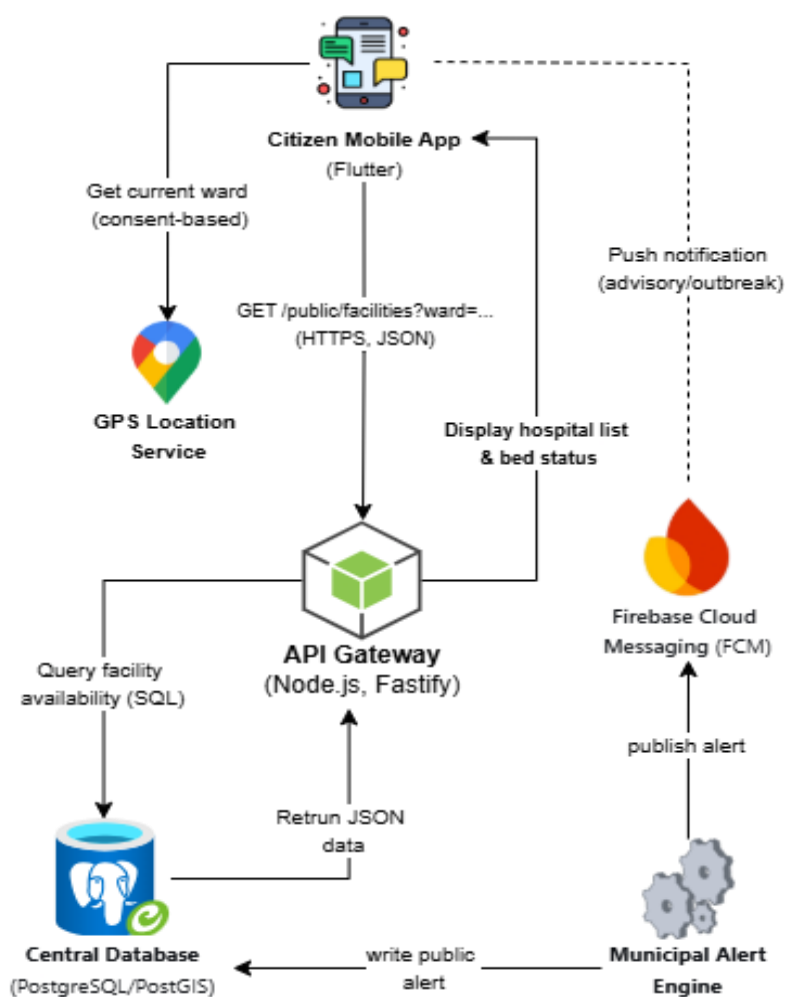


Figure 3: Citizen Engagement Architecture –

Public Access to Real-Time Health Information

System Overview

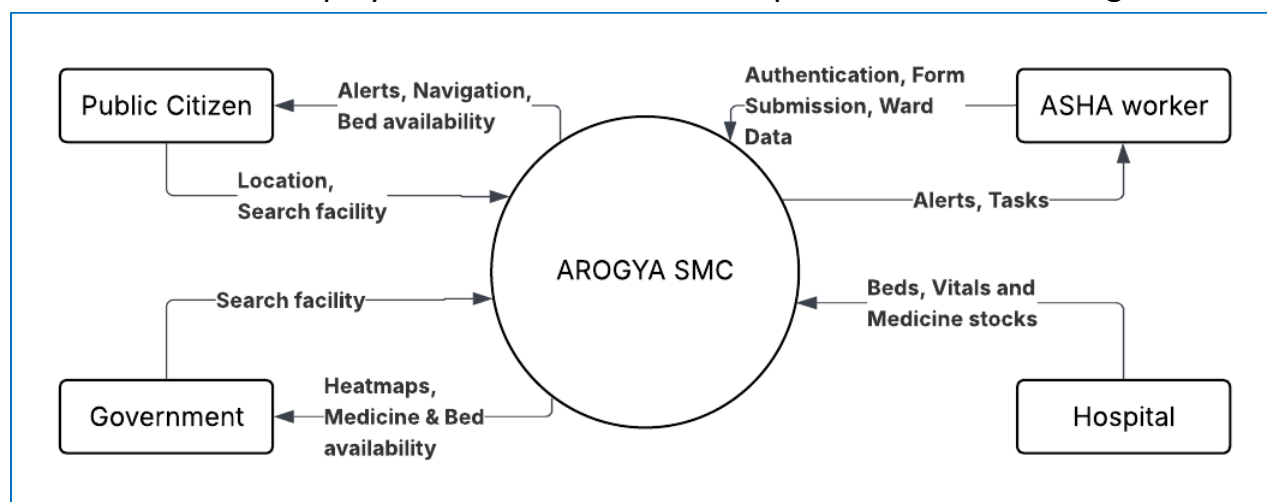
Arogya-SMC follows a modular, multi-stakeholder architecture designed for health data collection, standardization, analytics, and decision support. Field-level data is captured through the ASHA app, hospital data is submitted through the hospital portal, and public health information is accessed through the citizen app. All system requests pass through a secure API layer, where data is validated, standardized, stored, and processed for analytics and reporting.

The architecture is designed to support offline field collection, structured health data handling, ward-level intelligence, and real-time decision support for municipal health administration.

SAMVED-2026 prototype submission format

Data Flow

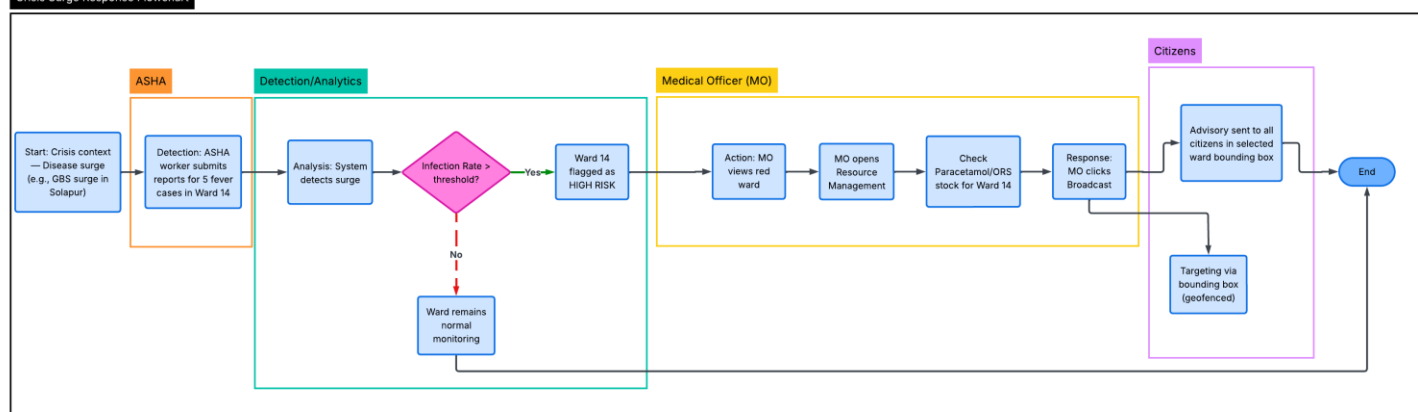
1. ASHA workers and hospitals enter health-related data
2. Data is transmitted to the secure API gateway
3. The FHIR standardization layer converts raw input into structured records
4. Standardized data is stored in PostgreSQL/PostGIS
5. Analytics engine processes the data for trends, alerts, and risk scoring
6. Results are displayed on the dashboard and pushed to users through notifications



User Flow

- ASHA worker submits field observations from the mobile app
- Hospital staff updates capacity and disease-related records
- Citizens view advisories, facilities, and alerts
- Municipal officers monitor dashboards, ward trends, and alerts for action

Crisis Surge Response Flowchart



SAMVED-2026 prototype submission format

Key Components

Module 1: Data Ingestion Hub (The Field Sensor)

This component is implemented primarily within the Flutter mobile application. It is optimized for real-world field conditions and enables efficient, structured data collection.

Key Features

- **S-P-LFormParser**
Converts standard IDSP (Integrated Disease Surveillance Programme) forms into structured database entries for seamless processing.
- **GeotaggingModule**
Automatically captures latitude and longitude for every report. This enables identification of micro-clusters at a granular level, such as specific streets or slum areas.

Module 2: Risk Intelligence Engine (The Brain)

Built using Next.js and PostgreSQL, this module transforms raw data into actionable insights for decision-makers.

Key Features

- **RiskScoringAlgorithm**
Utilizes the `v_ward_risk_intelligence` database view to compute a risk score (0–100) for each ward based on factors such as case velocity and population density.
- **EpidemiologicalCurveGenerator(Epi-Curve)**
Automatically generates 7-day and 30-day epidemiological curves, allowing Medical Officers (MO) to determine whether an outbreak is escalating or subsiding.

Module 3: Infrastructure Registry (The Supply Chain)

This module manages healthcare infrastructure and resource allocation, ensuring that medical supplies align with disease spread.

Key Features

- **InventoryBurn-RateEngine**
Implements SQL-based logic to calculate consumption rates. It predicts when essential supplies (e.g., ORS, Paracetamol) will be depleted in each ward.
- **LiveBedTracker**
Provides real-time visibility into ICU and ventilator availability across Primary Health Centers (PHCs) and civil hospitals.

Module 4: Governance & Push Vault (The Integrity Layer)

SAMVED-2026 prototype submission format

This module ensures transparency, accountability, and effective communication with both authorities and citizens.

Key Features

- SystemAuditLogTracker**
 Records all user activities along with IP addresses (e.g., “MO viewed Ward 14”). This ensures accountability and supports post-crisis government audits.
- FCMBroadcastDispatcher**
 Uses Firebase Cloud Messaging to send targeted alerts. In case of a localized outbreak, only citizens within the affected ward receive critical advisories (e.g., “Red Alert”).

5. Subsystems

Subsystem Name	Function (one or more)	Input	Output	Technology Used
ASHA Field Surveillance Module	- Collects field-level health data (symptoms, cases, visits) - Captures syndromic indicators (fever, cough, diarrhoea) - Reports maternal & child risk flags - Enables geotagged environmental observations - Receives advisories and alerts from system	Field observations, patient visit data (manual entry)	Syndromic reports to API Gateway; receives advisories/alerts	Mobile App (Flutter), SQLite (offline), HTTPS, JSON, JWT, FCM
Citizen Health Access & Engagement App	- Provides public access to health advisories - Enables facility and bed availability lookup - Displays outbreak alerts and notifications - Supports location-based services (nearest facility) - Delivers push	User queries (public), app interactions	Public queries to API Gateway; receives advisories, alerts, notifications	Mobile/Web App (Flutter), HTTPS, JSON (no auth), GPS, FCM

SAMVED-2026 prototype submission format

	notifications to citizens			
Hospital Capacity & Disease Reporting Portal	• Reports hospital capacity (beds, ICU, ventilators, oxygen) • Submits disease admission and occupancy data • Maintains historical logs and audit records • Provides secure role-based access for hospital staff	Capacity reports, disease case data	Capacity reports to API Gateway; receives history/confirmation responses	Web Portal (React, TypeScript), HTTPS, JSON, JWT, RBAC
Municipal Health Command Centre	• Monitors dashboards and ward-level heatmaps • Analyzes epidemiological trends • Tracks alerts and outbreak situations • Supports decision-making and resource planning • Performs drill-down analysis	Dashboard queries, admin requests	Queries to API Gateway; receives aggregated dashboard data (trends, heatmaps, alerts)	Web Dashboard (React, Leaflet, Recharts), HTTPS, JSON, JWT
Secure API Gateway & Ingestion Layer	• Acts as central entry point for all subsystems • Validates JWT and enforces security policies • Routes requests to appropriate services • Handles public and private endpoints • Sends push notifications (FCM) • Logs and manages all requests	All incoming requests from subsystems (reports, queries, alerts)	Sends data to standardization service, DB queries, FCM notifications; dashboard responses	Node.js (Fastify/Express), HTTPS, JSON, JWT, FCM, Redis
FHIR-Compliant Data Standardization Service	• Converts raw health data into FHIR	Raw reports from API Gateway	Standardized FHIR JSON (JSONB) stored in DB	Node.js, FHIR Mapping, JSONB

SAMVED-2026 prototype submission format

	Observations • Maps data to SNOMED CT / ICD-11 standards • Ensures interoperability and validation • Enriches data with ward/facility metadata • Normalizes and structures health records			processing, HTTP / Message Queue
Centralized Health Data Repository (PostgreSQL/PostGIS)	• Stores standardized health data (observations, capacity) • Maintains ward boundaries and facility registry • Supports geospatial queries (heatmaps) • Enables time-series health analytics • Acts as single source of truth	Standardized FHIR data; query requests from API Gateway	Query results; master data; bulk export for analytics	PostgreSQL, PostGIS, SQL, JSONB
Predictive Analytics & Alert Engine	• Performs data aggregation and anomaly detection • Models outbreak trends (time-series analysis) • Generates threshold-based alerts • Computes risk scores and forecasts • Publishes real-time alerts	Historical data from database	Alerts, aggregated metrics; publishes alerts to API Gateway via queue	Python, Pandas, Prophet, SQL, Redis Queue

SAMVED-2026 prototype submission format

6. Subsystem Interaction Matrix

Subsystem A	Subsystem B	Type of Interaction	Description
ASHA Field Reporting Layer	Secure API Gateway	Data / API	Sends structured field reports, symptoms, and geotagged entries for validation and processing
ASHA Field Reporting Layer	Data Standardization & Storage Layer	Data	Local reports are converted into structured records and stored for further analysis
Secure API Gateway	Analytics & Alert Engine	Control / Data	Triggers analytics routines when new data is available and forwards processed results
Data Standardization & Storage Layer	Risk Intelligence Engine	Data	Provides stored ward-wise health records for scoring, trend analysis, and anomaly detection
Data Standardization & Storage Layer	Infrastructure Registry	Data	Supplies health and resource data used for burn-rate and demand estimation
Risk Intelligence Engine	Visualization & Decision Support Layer	Data	Sends ward risk scores, outbreak trends, and alerts to dashboards
Infrastructure Registry	Visualization & Decision Support Layer	Data	Sends bed availability, inventory status, and resource forecasts to user interfaces
Governance & Push Vault	Citizen App / ASHA App / Dashboard	Control / API	Dispatches alerts, advisories, and notifications to relevant users
Secure API Gateway	Governance & Push Vault	Control / Data	Logs user actions and enforces secure, traceable communication

7. Prototype Description Workflow:

1. ASHA Field Surveillance App

Purpose: Field-level health data collection with offline-first capability

INPUT

- Syndromic data (fever, cough, diarrhea, jaundice, etc.)
- Maternal and child risk indicators (high-risk pregnancy, malnutrition)
- Environmental observations (stagnant water, sanitation issues)
- GPS coordinates (latitude, longitude)
- Optional image/photo evidence

PROCESSING

- Local persistence using offline-first architecture (SQLite)
- Geolocation tagging for ward-level mapping
- Data queued and synced automatically on connectivity restoration

OUTPUT

- Structured reports submitted to backend
- Sync status indicators (Pending / Synced / Failed)
- Local history and summary views

SAMVED-2026 prototype submission format

22:57

Register New Family

Basic Information

Family Name

Head of Family Name

Total Family Members

1

Contact Details

Phone Number

Ward

Ward 1 - North

Full Address

Register Family

22:53

Step 1 of 4

Health Cases

Health Cases

Select ward and enter observed cases

Ward 12 - Central

Fever

4

Cough

1

Diarrhea

1

Jaundice

3

Next

22:53

Step 2 of 4

Risk Flags

Risk Flags

Mark any identified health risks

Maternal Risk Flags

High Risk Pregnancy

Post-natal Complications

No ANC Visit

Child Risk Flags

Malnourished

Immunization Overdue

Previous

Next

2. Hospital Reporting Portal

Purpose: Infrastructure and capacity reporting

INPUT

- Bed availability (general, ICU, ventilators)
- Oxygen availability and emergency status
- Disease case updates and admissions
- Resource inventory data

PROCESSING

- Secure API validation (JWT + role-based access)
- Data normalization and structured storage

OUTPUT

- Capacity utilization metrics
- Resource status indicators
- Updated facility-level records

SAMVED-2026 prototype submission format

Routine Capacity Tracking

Please update your facility's bed and resources capacity regularly.

General Beds

Total Beds

100

Available Beds

50

ICU Capacity

Total ICU Beds

4

Available ICU Beds

2

Ventilators

Total Ventilators

2

Available Ventilators

1

Oxygen Status

☒ Oxygen Supply Available

Inventory Management

Secure Session

SOS Override

Inventory Management

Track and update your facility medical resources.

New inventory item added successfully!

RESOURCE TYPE	CURRENT STOCK	MIN THRESHOLD	LAST UPDATED	ACTIONS
RDT Kit Low	90	100	22/03/2026, 19:58:45	<button>Save</button> <button>Dispatch</button>
Sanitizer (Liters) Low	11	100	22/03/2026, 21:03:28	<button>Save</button> <button>Dispatch</button>

Add New Resource

Resource Type

Select Resource... ▼

Current Stock

0

Min Threshold

100

Add Item

SAMVED-2026 prototype submission format

3. Municipal Command Dashboard

Purpose: Analytics-driven decision support

INPUT

- Aggregated ASHA field data
- Hospital capacity and resource data
- Historical time-series records (7-day / 30-day windows)
- Ward-level demographic context

PROCESSING

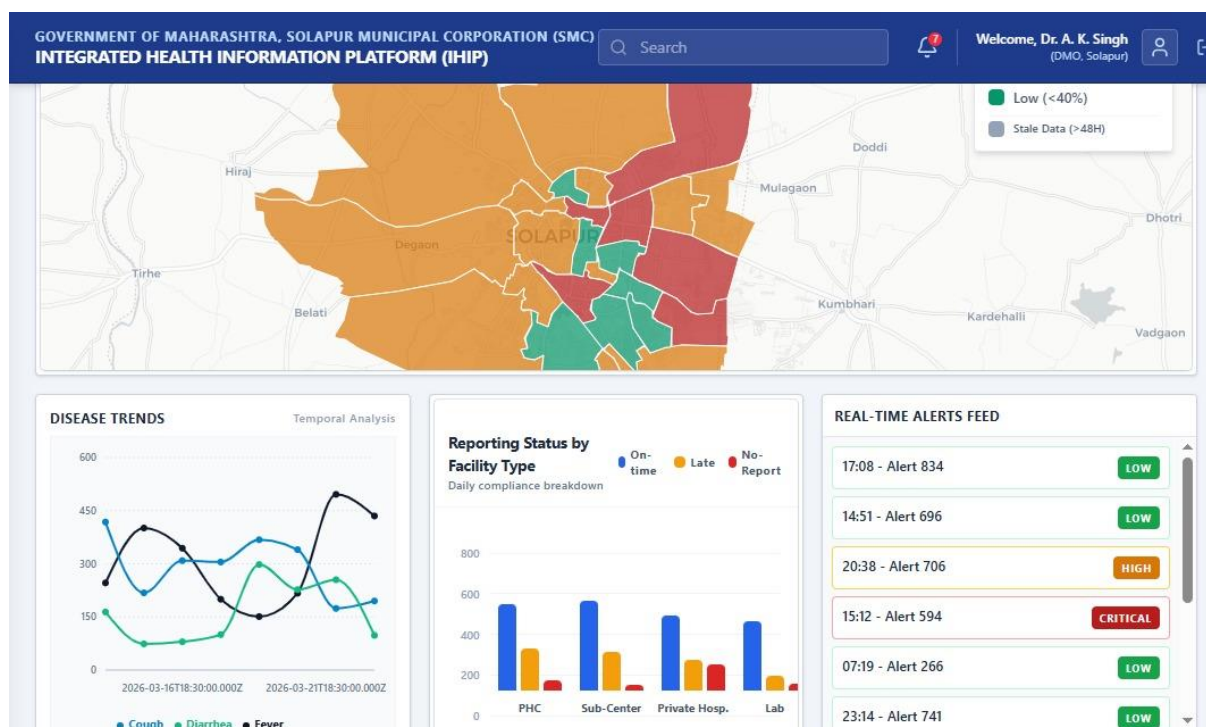
- Data aggregation and spatial grouping
- Statistical anomaly detection:

Threshold = $\mu + 2\sigma$ (rolling baseline)

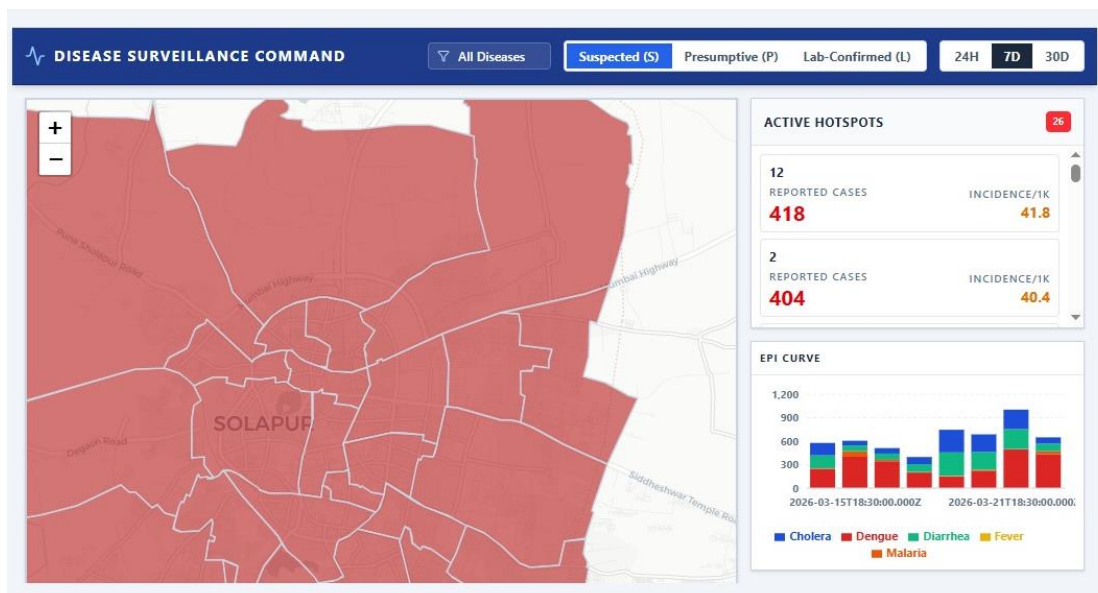
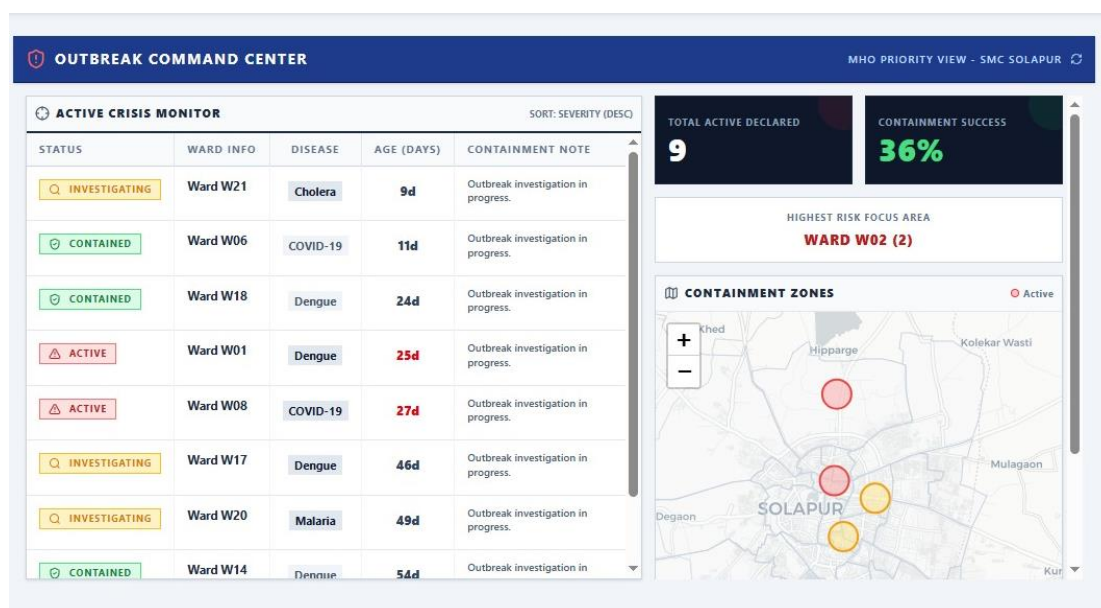
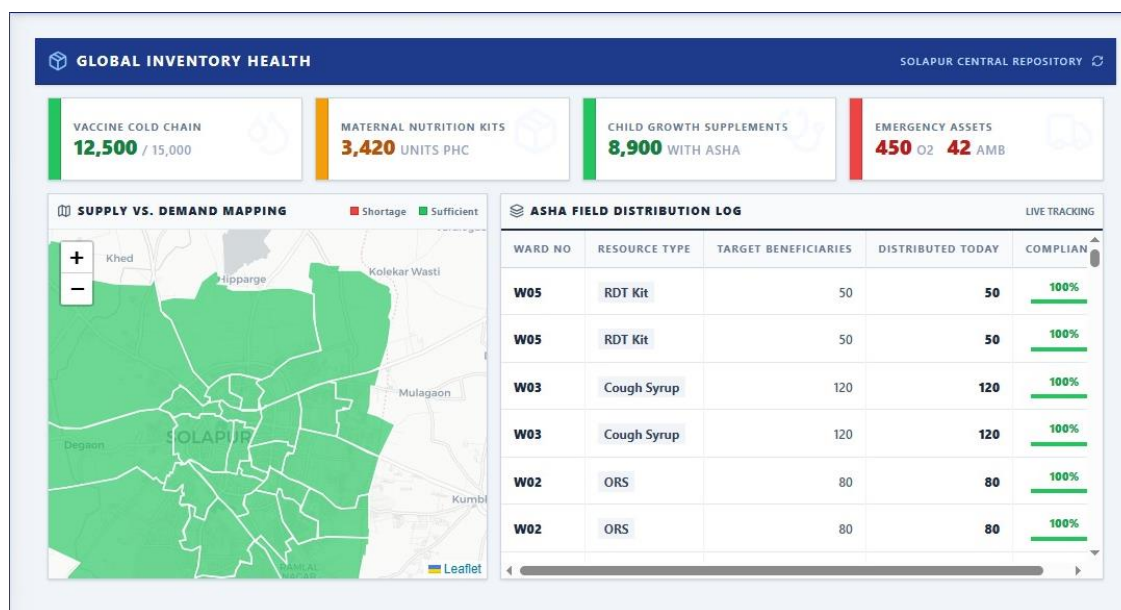
- Trend analysis and regression-based forecasting
- Risk scoring based on case intensity and trends

OUTPUT

- Ward-level risk visualization
- Outbreak alerts and anomaly flags
- Trend graphs and decision metrics



SAMVED-2026 prototype submission format



SAMVED-2026 prototype submission format

4. Citizen Health Access App

Purpose: Public access to health services

INPUT

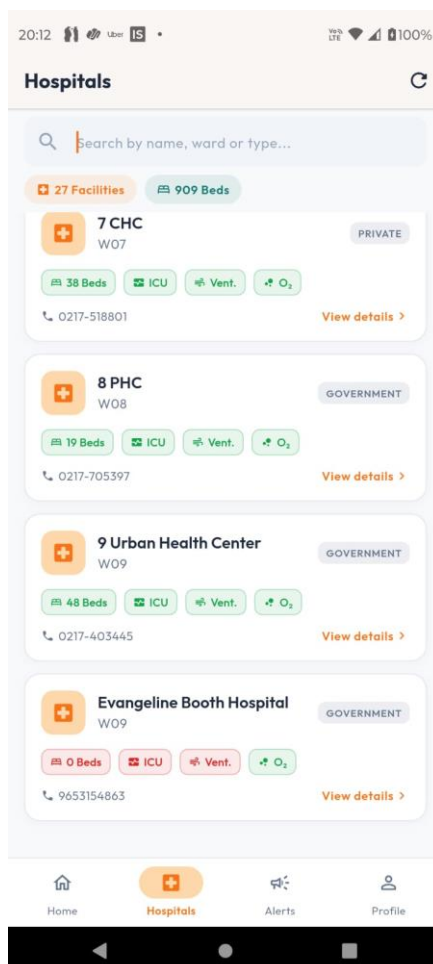
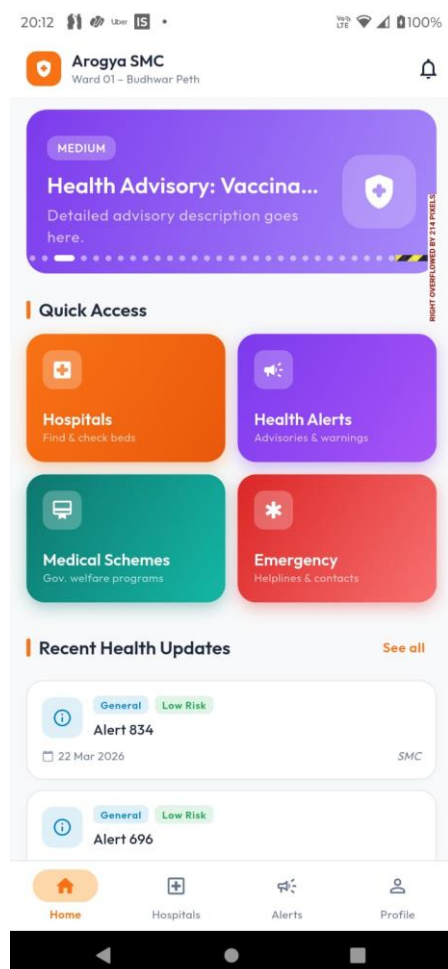
- User location (GPS)
- User queries (facility search, refresh actions)

PROCESSING

- Ward-level filtering of relevant data
- Cached responses for faster access
- Notification routing based on location

OUTPUT

- Hospital availability and details
- Health advisories and alerts
- Emergency service access



SAMVED-2026 prototype submission format

8. Technology Stack & Tools

Layer	Technology Used (Across all Subsystems)	Justification
Frontend	Mobile (ASHA & Citizen Apps): <i>Flutter, Riverpod, easy_localization</i> Web (Dashboard & Portal): <i>React, Next.js, Tailwind CSS, shadcn/ui</i> Visualization: <i>Leaflet, react-leaflet, leaflet.heat, Recharts</i>	Flutter enables a cross-platform, offline-first mobile experience optimized for real-world field conditions. React/Next.js supports modular and scalable UI development. Leaflet and Recharts enable spatial and temporal visualization , including ward-level heatmaps and trend analysis.
Backend	API Layer: <i>Node.js, Next.js API Routes</i> Language: <i>TypeScript</i> Data Handling: <i>REST APIs, JSON-based communication</i>	A unified backend layer handles secure data flow, validation, and routing across subsystems. TypeScript improves reliability and maintainability. The API-driven architecture enables seamless integration between mobile, web, and analytics components.
Database	Primary Database: <i>PostgreSQL (JSONB support)</i> Spatial Engine: <i>PostGIS</i> Local Storage: <i>Drift (SQLite), Hive</i>	PostgreSQL provides a robust and scalable data store. JSONB allows flexible handling of health records. PostGIS enables ward-level geospatial processing and mapping . Local storage ensures offline-first data persistence in field applications.
AI / ML	Analytics Engine: <i>Python, pandas, NumPy</i> Anomaly Detection: <i>Statistical thresholding ($\mu + 2\sigma$)</i> Forecasting: <i>Linear Regression (time-series trend modeling)</i>	The analytics layer performs data aggregation, anomaly detection, and short-term forecasting . Statistical models detect abnormal spikes, while regression-based forecasting predicts trends in disease spread and resource demand. This enables data-driven decision support at ward level.
Tools / Platforms	Authentication & Security: <i>Firebase (JWT)</i> Notifications: <i>Firebase Cloud Messaging (FCM)</i> System Support: <i>Redis, Geolocation APIs</i> Mapping Data: <i>SMC GIS datasets</i>	Firebase supports secure authentication and notification delivery. FCM enables targeted alert broadcasting . Redis supports system-level processing and queuing. GIS datasets enable accurate ward mapping and geospatial validation.

SAMVED-2026 prototype submission format

9. Innovation Highlights

➤ What is new in your solution?

- **End-to-end data flow (field → system → dashboard)**
Unlike many existing municipal systems that mainly display aggregated data, this solution captures primary data directly from ASHA workers and hospitals.
- **Offline-first field data collection**
While most existing platforms depend on continuous internet connectivity, this system allows offline data entry with later synchronization, making it practical for field conditions.
- **Ward-level micro-surveillance**
Existing dashboards generally provide city-level insights, whereas this system enables fine-grained ward-wise monitoring of diseases and risks.
- **Unified multi-stakeholder ecosystem**
Unlike separate systems (citizen apps, dashboards, hospital portals), this solution combines ASHA interface, hospital portal, citizen app, and government dashboard into one workflow.

➤ How is it better than existing approaches?

- Systems used during COVID-19 (such as municipal dashboards and smart city control rooms) were primarily visualization tools for already collected data, whereas this solution adds structured data collection at the ground level (ASHA + hospitals).
- Existing approaches follow a top-down model, where data is processed centrally and then shown on dashboards.
→ This system follows a bottom-up approach, starting from field data collection and moving upward to decision-making.
- Many existing systems rely on manual or delayed reporting, which can slow down response time.
→ This system enables faster and more structured data entry, improving responsiveness.
- Current platforms often lack granular, ward-level intelligence.
→ This system supports zone-wise and ward-level insights, enabling targeted interventions.

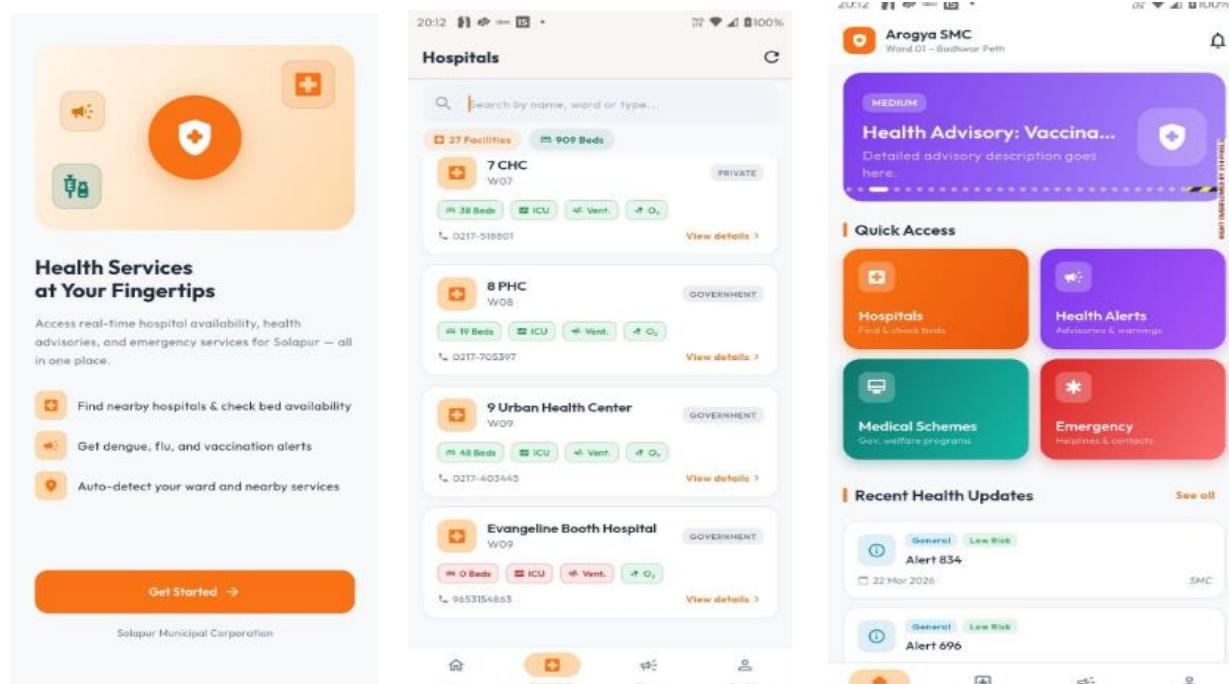
➤ Any AI / data-driven innovation?

- Basic anomaly detection logic to identify sudden increases in disease cases based on incoming data.
- Experimental Python-based prediction module to analyze trends and provide early signals for potential outbreaks.
- Data-driven dashboards that support visualization of trends, risk zones, and decision-making for municipal authorities.

SAMVED-2026 prototype submission format

10. Results / Demonstration

Citizen app – OUTPUT:



User opens app → Splash screen → clicks Get Started.

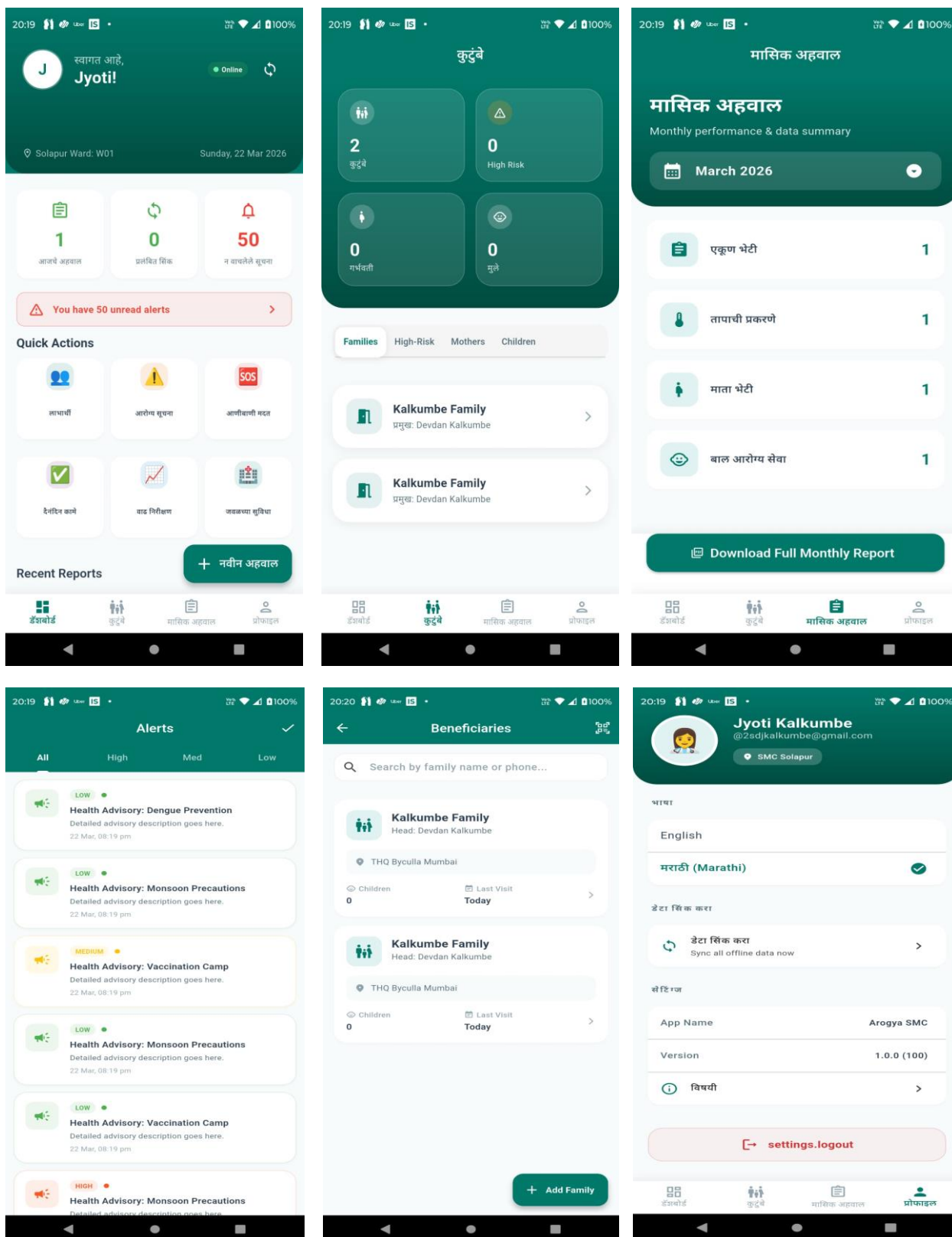
Lands on Home screen → views health advisory carousel and quick access options Selects Hospitals → views hospital list with bed availability and contact details.

Selects Health Alerts → views important alerts and advisories.

Navigates between sections to access required health information.

ASHA WORKER APP OUTPUT:

SAMVED-2026 prototype submission format



ASHA worker opens app → logs in → dashboard loads → app sends request to API Gateway → API Gateway fetches worker data (ward, stats, alerts) from database → sends response → dashboard displayed with visits, alerts, and quick actions

SAMVED-2026 prototype submission format

ASHA worker selects “**Families/Beneficiaries**” → app sends request to API Gateway → API Gateway queries PostgreSQL database → retrieves family records → sends response → family list displayed in app

ASHA worker adds or updates beneficiary data → enters details (symptoms, visits, risk flags, etc.) → data stored locally in SQLite (offline mode) → when sync is triggered → app sends data to API Gateway (HTTPS, JSON, JWT) → API Gateway validates request → forwards raw data to FHIR Standardization Service → service converts data into FHIR-compliant format (SNOMED CT / ICD-11) → standardized data stored in PostgreSQL (JSONB)

ASHA worker opens “**Monthly Report**” → app sends request to API Gateway → API Gateway performs aggregate queries on database → retrieves monthly statistics → sends response → report displayed/downloaded in app

Analytics Engine reads stored health data from database → performs analysis and trend detection → generates alerts and risk scores → sends alerts to API Gateway → API Gateway pushes alerts via FCM → ASHA app receives alerts and displays them

- Metrics (if applicable): accuracy, time, efficiency, etc.

11. Reproducibility & Code Access

Code Access

The Arogya-SMC system is organized into modular repositories representing different subsystems:

Core Platform (Backend + Dashboard + Analytics Integration)

<https://github.com/Stavan1234/arogya-smc-platform>

ASHA Reporting Backend

<https://github.com/Stavan1234/arogya-asha-app-backend>

Citizen / Public Backend

<https://github.com/Stavan1234/arogya-public-app-backend>

ASHA Mobile Application

<https://github.com/Stavan1234/arogya-smc-ASHA-app>

Citizen Mobile Application

<https://github.com/Stavan1234/arogya-smc-public-app>

System Requirements

To reproduce the prototype, the following environment stack is required:

SAMVED-2026 prototype submission format

- **Database:** PostgreSQL with PostGIS extension
- **Backend:** Node.js with TypeScript support
- **Mobile:** Flutter SDK (3.x recommended)
- **Analytics Engine:** Python 3.x (pandas, NumPy, scikit-learn)
- **Other Tools:** Firebase (Authentication & Notifications)

Environment Configuration

Before running the system, configure the following environment variables:

- JWT_SECRET → authentication token signing
- NOTIFICATION_TRIGGER_SECRET → secure alert triggering
- FIREBASE_SERVICE_ACCOUNT → Firebase Admin SDK configuration
- Database credentials → PostgreSQL connection setup

Deployment Workflow

Step 1: Database Setup

- Install PostgreSQL and enable PostGIS
- Create database and required tables
- Load initial schema and seed data

Step 2: Backend Initialization

- Navigate to the **core platform repository**
- Install dependencies:
npm install
- Start server:

npm run dev

This launches API services, dashboards, and core backend modules.

Step 3: Analytics Engine Execution

- Navigate to the analytics directory
- Install dependencies:
pip install pandas numpy scikit-learn
- Run analytics pipeline:

python run_all.py

This executes:

- anomaly detection
- outbreak forecasting

SAMVED-2026 prototype submission format

- resource demand prediction

Step 4: Mobile Application Setup

Navigate to Flutter app directory

- Install dependencies:

flutter pub get

- Run application:

flutter run

Network Configuration (Important)

When testing on a physical device():

adb reverse tcp:3001 tcp:3001

Ensure API base URL points to:

http://localhost:3001/api

System Reproducibility

The system can be reproduced by following the above steps, enabling:

- End-to-end data flow from field input to dashboard
- Execution of analytics pipeline on real or simulated data
- Functional demonstration of all major subsystems

12. SMC Impact Assessment (Proposed)

Impact Area	Current State	Proposed Improvement	Expected Benefit
Time Efficiency	Manual, delayed data collection	Digital data entry and dashboard visualization	Faster decision-making & response
Disease Detection	Late identification of outbreaks	Basic alert generation and trend monitoring	Reduced disease spread & risk
Resource Utilization	Inefficient allocation of resources	Zone-wise data-driven allocation	Optimized use of beds, staff, and medicines
Operational Efficiency	Fragmented systems & manual processes	Modular digital system (apps + dashboard)	Improved workflow & reduced errors
Citizen Satisfaction	Limited access & delayed services	Citizen mobile app for information and alerts	Increased accessibility and satisfaction

13. Limitations & Future Scope

➤ Current Limitations

- The system is currently developed as separate modules, and full integration into a single centralized backend is still in progress.
- Data is stored in local PostgreSQL databases on individual systems rather than a unified cloud-based database.
- Real-time data flow between ASHA workers, hospitals, and the government dashboard is partially simulated using locally stored data.
- The hospital portal is in prototype stage and not fully connected to other modules.
- The analytics and prediction module is at an initial stage, using basic logic rather than a fully trained model.
- The data ingestion layer, security mechanisms (JWT/auth), and FHIR-based standardization layer are part of the proposed architecture but are not fully implemented in the current prototype.

➤ Assumptions Made

- It is assumed that ASHA workers and hospitals will regularly update data in the system.
- Reliable internet connectivity is assumed during data synchronization (ASHA app sync feature).
- The data used in the prototype represents realistic scenarios for demonstration purposes.
- It is assumed that citizens and government officials will actively use the system for decision-making.

➤ Possible Future Improvements

- Integration of all subsystems into a single centralized backend and database for real-time data flow.
- Implementation of a secure API gateway with authentication (JWT) and data validation mechanisms.
- Deployment of a FHIR-compliant data standardization layer to ensure interoperability with healthcare systems.
- Deployment on cloud infrastructure for scalability and city-wide accessibility. Enhancement of the analytics module using advanced machine learning models for accurate disease prediction.
- Implementation of real-time notifications and alert systems using live data streams.
- Integration with national health systems such as ABDM/IDSP for interoperability.
- Improvement of the hospital portal with full data validation and real-time updates

SAMVED-2026 prototype submission format

Team Information					
Team name: Beast Innovators			Institute code: 1-4849231		
Institute name: Agnel Charities Fr. C. Rodrigues Institute of Technology.					
City: Vashi, Navi Mumbai			State: Maharashtra		
Team Leader Information					
Leader name: Suman Bhaskar Bhowmik					
Contact number: 7400167998			Email: bhowmiksuman4545@gmail.com		
Mentor Details					
Mentor name: Dr. Pravin S. Rahate					
Contact number: 9833971055			Email: pravin.rahate@fcrit.ac.in		
Participant Details					
SN	Participant name	Program	Sem	Contact number	Email
1	Suman Bhowmik	Computer Engineering	Sem 6	7400167998	bhowmiksuman4545@gmail.com
2	Aaron Antony	Computer Engineering	Sem 6	9323201460	aaronantonytheofficial@gmail.com
3	Stavan Kalkumbe	Computer Engineering	Sem 6	9653154863	stavankalkumbe@gmail.com
4	Bipin P Kuruvilla	Computer Engineering	Sem 6	9137406241	bipinkuruvilla134@gmail.com
5	Raghav Sharma	Computer Engineering	Sem 6	8169018774	raghav.a.sharma34@gmail.com

Declaration

We hereby declare that the work titled “**Smart Health Solutions for Solapur Municipal Corporation.**” submitted for **SAMVED-2026** is the original outcome of our team’s efforts.
The work has been carried out by us under the guidance of the undersigned mentor and **is neither copied from any source nor generated using artificial intelligence (AI) tools.**

Place: Vashi

Date: 22/03/2026

Team Leader Name: Suman Bhaskar Bhowmik

Mentor Name: Dr. Pravin S. Rahate